PROGRESS III
KONTROL MOTOR NIRKABEL DENGAN OTOMASI TIMER COUNTDOWN

3.1 Tujuan Modul

Tahap Progress 3 bertujuan untuk mengembangkan sistem otomasi hitung mundur (*countdown timer*) yang diintegrasikan ke dalam sistem kendali motor nirkabel Blynk. Pengujian difokuskan pada pengolahan algoritma *dynamic timer* berbasis waktu internal mikrokontroler (menggunakan fungsi millis()), sinkronisasi input waktu (jam, menit, detik) dari antarmuka *smartphone*, serta kemampuan ESP32 dalam memutus arus secara mandiri dan akurat begitu durasi waktu habis.

3.2 Skema Rangkaian

Skema pengkabelan pada tahap ini tidak mengalami perubahan fisik dari tahap sebelumnya. ESP32 tetap dihubungkan ke modul *driver* motor L298N dengan konfigurasi:

- Pin 27 (motor1Pin1): Dihubungkan ke pin IN1 *driver* motor.
- Pin 26 (motor1Pin2): Dihubungkan ke pin IN2 *driver* motor.
- Pin 12 (enable1Pin): Dihubungkan ke pin ENA *driver* motor (untuk sinyal PWM).

3.3 Konsep & Logika Sistem

Sistem pada modul ini memperkenalkan paradigma *Non-Blocking Delay* (penundaan tanpa memblokir sistem). Jika menggunakan perintah delay() biasa, mikrokontroler akan "membeku" dan tidak bisa menerima perintah lain. Oleh karena itu, program menggunakan metode evaluasi waktu berbasis millis() (menghitung selisih waktu berjalan sistem dalam milidetik).

Pengguna dapat mengatur input Jam, Menit, dan Detik melalui *Slider* di aplikasi Blynk. ESP32 kemudian akan mengkalkulasinya menjadi total durasi milidetik. Selama fungsi *timer* berjalan, ESP32 akan terus memonitor waktu secara *background* sembari mengirimkan pembaruan teks (string format HH:MM:SS) ke layar pengguna. Begitu target durasi tercapai, ESP32 memicu interupsi internal untuk mematikan putaran motor.

3.4 Konfigurasi Widget

Tahap ini menggunakan ulang perangkat (*Device*), *Template*, dan kredensial *Auth Token* yang telah dikonfigurasi secara utuh pada Progress 2. Oleh karena itu, proses instansiasi perangkat baru tidak lagi diperlukan. Fokus modifikasi arsitektur pada tahap ini hanya terletak pada penambahan

komponen antarmuka pengguna (*User Interface*) baru di menu *Developer Zone* aplikasi Blynk guna mengakomodasi fitur hitung mundur (*countdown timer*).

Berikut adalah penjabaran detail mengenai konfigurasi penambahan *widget* yang dipetakan secara spesifik ke dalam *Datastream* mikrokontroler:

1. Konfigurasi Pengatur Jam (HOURS SLIDER & HOURS):
    - Jenis Widget: 1 buah *Slider* dan 1 buah *Value Display*.
    - Datastream: Kedua *widget* tersebut ditautkan secara paralel pada V4 (HOURS SLIDER) dengan tipe data *Integer*. Atur rentang nilai minimal 0 dan maksimal 24.
    - Fungsi & Penamaan: *Slider* bertindak sebagai masukan analog untuk menetapkan durasi jam atur parameter *Title* menjadi "HOURS SLIDER". Sementara itu, *Value Display* berfungsi untuk menampilkan angka presisi dari posisi *slider* saat digeser atur parameter *Title* menjadi "HOURS".

2. Konfigurasi Pengatur Menit (MINUTES SLIDER & MINUTES):
    - Jenis Widget: 1 buah *Slider* dan 1 buah *Value Display*.
    - Datastream: Keduanya ditautkan secara paralel pada V5 (MINUTES SLIDER) dengan tipe data *Integer*. Atur rentang nilai minimal 0 dan maksimal 59.
    - Fungsi & Penamaan: *Slider* bertindak sebagai instrumen masukan durasi menit, atur parameter *Title* menjadi "MINUTES SLIDER". *Value Display* secara waktu nyata menampilkan angka spesifik yang sedang dipilih, atur parameter *Title* menjadi "MINUTES".

3. Konfigurasi Pengatur Detik (SECONDS SLIDER & SECONDS):
    - Jenis Widget: 1 buah *Slider* dan 1 buah *Value Display*.
    - Datastream: Keduanya ditautkan secara paralel pada V6 (SECONDS SLIDER) dengan tipe data *Integer*. Atur rentang nilai minimal 0 dan maksimal 59.
    - Fungsi & Penamaan: *Slider* digunakan untuk mengatur parameter waktu dalam satuan detik, atur parameter *Title* menjadi "SECONDS SLIDER". *Value Display* memastikan pengguna dapat mengonfirmasi nilai masukan detik secara akurat atur parameter *Title* menjadi "SECONDS".

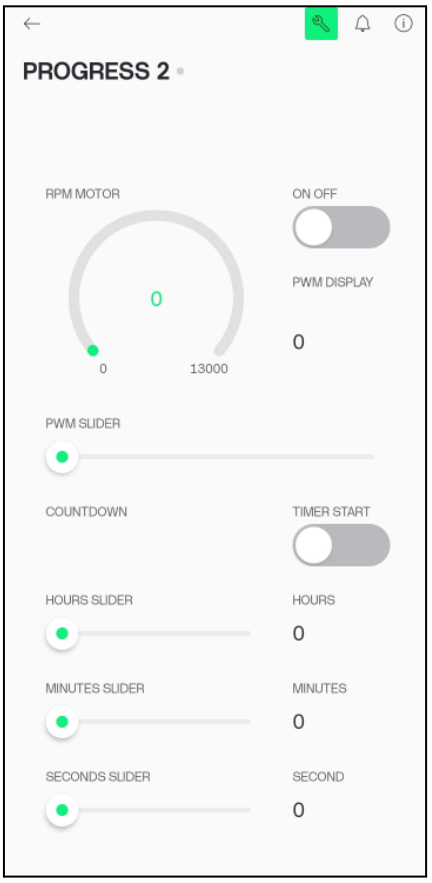
4. Konfigurasi Tombol Eksekusi (TIMER START):
    - Jenis Widget: *Button*
    - Datastream: Tautkan pada V7 (TIMER START) dengan tipe data *Integer* (0/1).
    - Parameter Mode & Penamaan: Ubah judul (*Title*) menjadi

"TIMER START".

- Fungsi: Tombol ini berfungsi sebagai pemicu yang menginstruksikan ESP32 untuk mengonversi nilai dari ketiga *slider* menjadi total milidetik dan mengeksekusi algoritma hitung mundur.

5. Konfigurasi Tampilan Layar Hitung Mundur (COUNTDOWN):
   - Jenis Widget: *Value Display*
   - Datastream: Tautkan pada V8 (COUNTDOWN). Berbeda dengan komponen sebelumnya, *Datastream* ini wajib dikonfigurasi menggunakan tipe data String.
   - Fungsi & Penamaan: Ubah judul (*Title*) menjadi "COUNTDOWN". *Widget* ini bertugas menerima data pemrosesan dari fungsi C++ sprintf() di dalam ESP32, lalu menampilkannya secara interaktif kepada pengguna dalam format teks dinamis "HH:MM:SS" (Jam:Menit:Detik) yang terus menyusut secara presisi setiap satu detik.

Setelah seluruh komponen widget tersebut selesai ditambahkan pada kanvas Developer Zone, aturlah posisinya sedemikian rupa agar tertata rapi dan ergonomis, seperti yang diilustrasikan pada Gambar 1 berikut:



Gambar 1. Konfigurasi dan integrasi tata

letak *widget dynamic timer* pada kanvas aplikasi Blynk.

3.5 Source Code dan Penjelasan Logika Program

```
// PROGRESS 3 : KONTROL MOTOR + DYNAMIC TIMER + BLYNK
// Fitur: ON/OFF, PWM, Hitung Mundur Kustom via Aplikasi Blynk

// --- KREDENSIAL BLYNK & WIFI ---
#define BLYNK_TEMPLATE_ID "ISI_DENGAN_TEMPLATE_ID_ANDA"
#define BLYNK_TEMPLATE_NAME "ISI_DENGAN_NAMA_TEMPLATE_ANDA"
#define BLYNK_AUTH_TOKEN "ISI_DENGAN_AUTH_TOKEN_DEVICE_BARU"
#define BLYNK_PRINT Serial

#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>

// Masukkan SSID dan Password Wi-Fi/Tethering Anda
char ssid[] = "NAMA_WIFI_ANDA";
char pass[] = "PASSWORD_WIFI_ANDA";

//  DOKUMENTASI VIRTUAL PIN PADA BLYNK
// V0 : Gauge   - Simulasi Putaran Mesin / RPM (0 - 13000)
// V1 : Gauge   - Tampilan Nilai Kecepatan PWM (0 - 255)
// V2 : Slider  - Pengatur Kecepatan Motor PWM (0 - 255)
// V3 : Button  - Sakelar Motor ON/OFF (Mode Switch, 0/1)
// V4 : Slider  - Input Jam (0 - 24)
// V5 : Slider  - Input Menit (0 - 59)
// V6 : Slider  - Input Detik (0 - 59)
// V7 : Button  - Tombol Start Timer (Mode Switch, 0/1)
// V8 : Display - Teks Hitung Mundur / Countdown (String)

// --- KONFIGURASI PIN ---
int motor1Pin1 = 27;    // Pin IN1 Driver Motor
int motor1Pin2 = 26;    // Pin IN2 Driver Motor
int enable1Pin = 12;    // Pin ENA (PWM) Driver Motor

// --- PENGATURAN PWM ---
const int freq = 30000;      // Frekuensi PWM
const int resolution = 8;    // Resolusi 8-bit (0-255)
int dutyCycle = 0;           // Kecepatan awal
bool isMotorRunning = false; // Status motor

// --- VARIABEL DYNAMIC TIMER ---
int inputJam = 0;
int inputMenit = 0;
int inputDetik = 0;

unsigned long waktuMulaiTimer = 0; // Mencatat waktu saat timer dimulai
unsigned long durasiTimer = 0;     // Total durasi timer dalam milidetik
```

```
bool isTimerAktif = false;         // Status timer berjalan
long sisaWaktuTerakhir = -1;       // Menyimpan detik terakhir (mencegah spam Blynk)

// Fungsi Kontrol Motor & Update Layar Blynk

// Menerapkan nilai PWM ke hardware & update antarmuka Blynk
void applyMotor() {
  if (isMotorRunning) {
    digitalWrite(motor1Pin1, HIGH);
    digitalWrite(motor1Pin2, LOW);
    ledcWrite(enable1Pin, dutyCycle);

    Blynk.virtualWrite(V1, dutyCycle); // Update gauge PWM

    int simulasiRPM = map(dutyCycle, 0, 255, 0, 13000);
    Blynk.virtualWrite(V0, simulasiRPM); // Update gauge RPM

  } else {
    digitalWrite(motor1Pin1, LOW);
    digitalWrite(motor1Pin2, LOW);
    ledcWrite(enable1Pin, 0);

    Blynk.virtualWrite(V1, 0);
    Blynk.virtualWrite(V0, 0);
  }
}

// Menerima perintah dari Tombol ON/OFF Manual (V3)
BLYNK_WRITE(V3) {
  if (param.asInt() == 1) {
    isMotorRunning = true;
    isTimerAktif = false; // Batalkan timer jika tombol manual ditekan
    if (dutyCycle == 0) dutyCycle = 150;
    Serial.println(">>> [BLYNK] Motor ON ");
  } else {
    isMotorRunning = false;
    isTimerAktif = false; // Batalkan timer jika dimatikan manual
    Serial.println(">>> [BLYNK] Motor OFF ");
  }
  applyMotor();
}

// Menerima perintah dari Slider Kecepatan (V2)
BLYNK_WRITE(V2) {
  dutyCycle = param.asInt();
  Serial.print(">>> [BLYNK] PWM Diubah menjadi: ");
  Serial.println(dutyCycle);
  if (isMotorRunning) applyMotor();
}
```

```
// Membaca nilai dari Slider Waktu (V4, V5, V6)
BLYNK_WRITE(V4) { inputJam = param.asInt(); }
BLYNK_WRITE(V5) { inputMenit = param.asInt(); }
BLYNK_WRITE(V6) { inputDetik = param.asInt(); }

// Menerima perintah dari Tombol Start Timer (V7)
BLYNK_WRITE(V7) {
  if (param.asInt() == 1) {
    // Kalkulasi total waktu dari slider ke dalam milidetik
    durasiTimer = (inputJam * 3600UL + inputMenit * 60UL + inputDetik) * 1000UL;

    if (durasiTimer > 0) {
      isTimerAktif = true;
      isMotorRunning = true;
      waktuMulaiTimer = millis();
      sisaWaktuTerakhir = -1; // Reset agar langsung update layar

      if (dutyCycle == 0) dutyCycle = 150;
      applyMotor();

      Serial.println("\n>>> [TIMER] Memulai hitung mundur dari aplikasi!");
    } else {
      // Jika tombol start ditekan tapi semua slider waktu masih 0
      Blynk.virtualWrite(V7, 0);            // Kembalikan tombol ke OFF
      Blynk.virtualWrite(V8, "Atur Waktu!"); // Tampilkan pesan peringatan
    }
  } else {
    // Jika tombol start dimatikan manual di tengah jalan
    isTimerAktif = false;
    isMotorRunning = false;
    applyMotor();
    Blynk.virtualWrite(V8, "BATAL");
    Serial.println(">>> [TIMER] Dibatalkan oleh pengguna.");
  }
}

// Setup Awal

void setup() {
  Serial.begin(115200);

  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  ledcAttach(enable1Pin, freq, resolution);

  isMotorRunning = false;
  applyMotor();
```

```
  Serial.println("\n Menghubungkan ke WiFi dan Blynk... ");
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);

  Serial.println(">>> Berhasil Terhubung ke Blynk Cloud!");
  Blynk.virtualWrite(V8, "00:00:00"); // Reset layar timer di HP saat baru menyala
}

// Loop Utama

void loop() {
  Blynk.run(); // Menjaga koneksi dengan server Blynk

  // --- LOGIKA HITUNG MUNDUR FORMAT JAM:MENIT:DETIK ---
  if (isTimerAktif) {
    unsigned long waktuBerjalan = millis() - waktuMulaiTimer;

    if (waktuBerjalan < durasiTimer) {
      long sisaTotalDetik = (durasiTimer - waktuBerjalan) / 1000;

      // Update layar HP hanya saat detiknya berubah agar aplikasi tidak lag
      if (sisaTotalDetik != sisaWaktuTerakhir) {
        int h = sisaTotalDetik / 3600;
        int m = (sisaTotalDetik % 3600) / 60;
        int s = sisaTotalDetik % 60;

        // Format teks menjadi 00:00:00
        char teksWaktu[10];
        sprintf(teksWaktu, "%02d:%02d:%02d", h, m, s);

        Blynk.virtualWrite(V8, teksWaktu);

        // --- Mencetak Teks Hitung Mundur ke Serial Monitor ---
        Serial.print(">>> [TIMER] Sisa Waktu: ");
        Serial.println(teksWaktu);

        sisaWaktuTerakhir = sisaTotalDetik;
      }
    } else {
      // Waktu Habis
      isTimerAktif = false;
      isMotorRunning = false;
      applyMotor();

      Blynk.virtualWrite(V7, 0);         // Matikan tombol Start di HP
      Blynk.virtualWrite(V8, "SELESAI"); // Tampilkan teks selesai
      Serial.println("\n>>> [TIMER] Waktu habis! Motor otomatis MATI.");
    }
  }
}
```

Penjelasan program

Inovasi utama pada tahap otomasi ini adalah pemanfaatan fungsi internal mikrokontroler millis() untuk merealisasikan waktu hitung mundur. Konsep ini mencegah terblokirnya eksekusi baris kode lain (berbeda dengan perintah delay() yang membekukan mikrokontroler). Blok BLYNK_WRITE() V4, V5, dan V6 secara pasif menangkap masukan nilai durasi jam, menit, dan detik dari antarmuka *slider* ponsel. Saat fungsi *trigger* pada V7 diaktifkan, program melakukan kalkulasi matematis (variabel durasiTimer) untuk mengonversi seluruh input waktu tersebut menjadi total satuan waktu berkaliber milidetik.

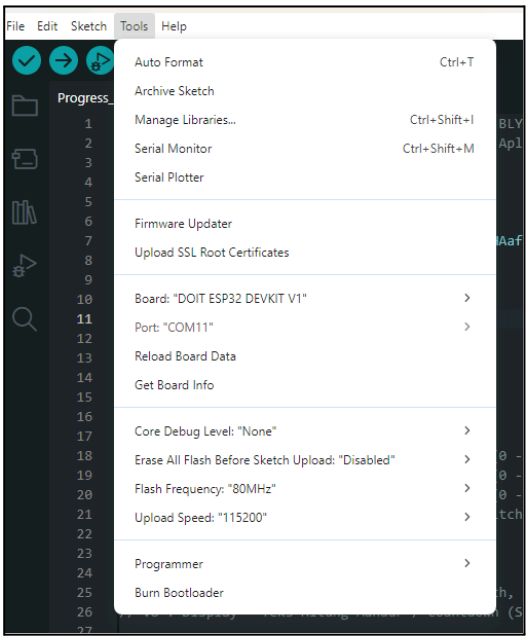
Mekanisme iteratif utama berjalan mulus di dalam siklus loop(). Variabel waktuBerjalan terus menghitung selisih antara waktu saat ini dengan waktuMulaiTimer menggunakan instrumen millis(). Guna mencegah masalah *flooding* (banjir data jaringan) pada server Blynk, mikrokontroler mengimplementasikan blok kondisi pembatas (if (sisaTotalDetik != sisaWaktuTerakhir)). Blok algoritma ini bertindak sebagai gerbang logika (*logic gate*) yang memastikan bahwa ESP32 hanya akan mengirimkan perintah Blynk.virtualWrite() ke layar antarmuka *smartphone* (V8) secara eksklusif setiap selang waktu tepat satu detik, dan bukan mengirim data secara terus-menerus setiap milidetik.

Pemrosesan teks waktu dilakukan dengan mendayagunakan fungsi bawaan C++ sprintf(). Fungsi ini secara dinamis memformat dan menyatukan hasil pembagian jam (h), sisa hasil bagi (*modulus*) menit (m), dan detik (s) ke dalam konfigurasi *string* dua digit standar ("00:00:00"). Begitu parameter waktuBerjalan dinyatakan melampaui rentang durasiTimer, arsitektur program secara instan menonaktifkan status variabel isTimerAktif dan isMotorRunning, menginterupsi aliran sinyal PWM fisik sehingga aktuator mesin berhenti secara drastis, lalu menonaktifkan posisi tombol pada layar *smartphone* untuk mensinkronkan ulang kondisi perangkat keras dengan perangkat lunaknya.

3.6 Hasil Pengujian & Analisis

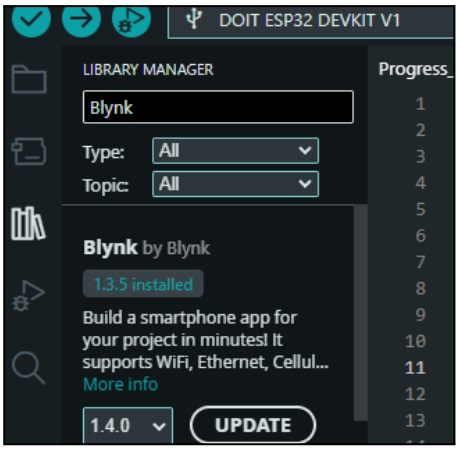
a. **Proses Kompilasi dan Upload Program**

Sebelum melakukan kompilasi dan *upload* program, pastikan terlebih dahulu bahwa *port* komunikasi sudah terhubung dengan benar ke kit ESP32. Pastikan jenis *board* yang dipilih adalah DOIT ESP32 DEVKIT V1 dengan kecepatan unggah (*Upload Speed*) sebesar 115200.



Gambar 2**.** Menu Tools Pemilihan Board dan Port pada Arduino IDE

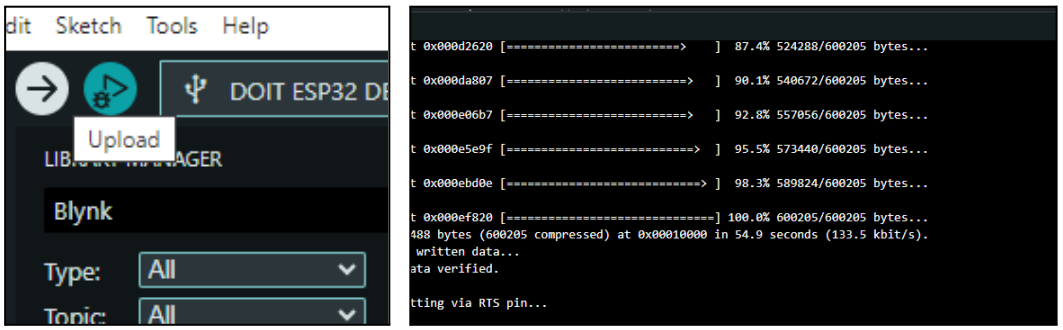
Selain itu, pastikan kembali bahwa *library* "Blynk by Blynk" telah terinstal pada *Library Manager*, sesuai dengan tahap persiapan (*setup*).



Gambar 3. Pengecekan Library

Blynk pada Arduino IDE

Hal krusial lainnya yang wajib dipastikan adalah jaringan Wi-Fi (atau *hotspot* seluler) yang digunakan untuk mengoneksikan ESP32 beroperasi pada pita frekuensi 2,4 GHz. Modul nirkabel pada ESP32 tidak mendukung konektivitas pada frekuensi 5 GHz. Jika pengaturan jaringan sudah dipastikan benar, langkah selanjutnya adalah menekan tombol *Upload* untuk menanamkan kode (*firmware*) ke dalam mikrokontroler.

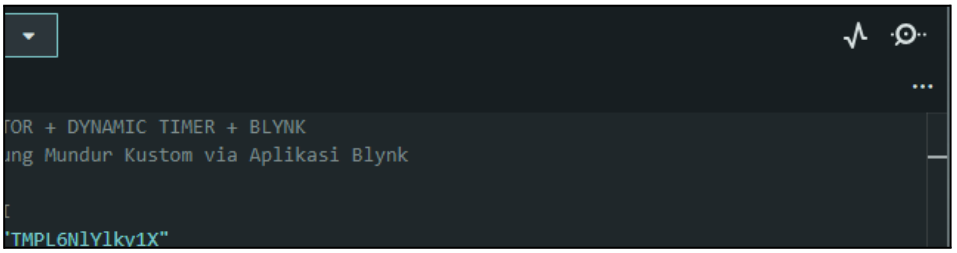


Gambar 4 dan 5. Proses Kompilasi
dan Upload Program

Tunggu hingga proses *upload* selesai. Jika kode berhasil diunggah tanpa galat (*error*), maka pada panel *Output* akan muncul log keterangan bahwa memori *flash* telah berhasil ditulis, ditandai dengan informasi "Wrote ... bytes" dan eksekusi *Hard resetting via RTS pin*.

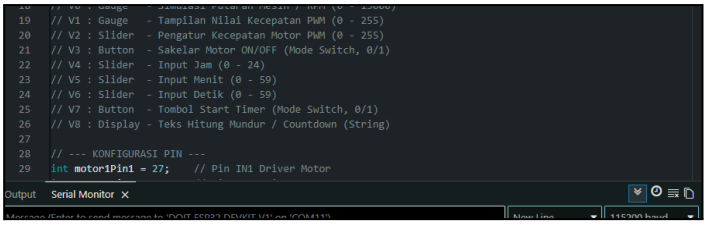
b. Pengujian Perintah via Serial Monitor & Blynk

Setelah program berhasil ditanamkan, pengujian fungsionalitas dan pemantauan (*monitoring*) dilakukan melalui antarmuka Serial Monitor. Langkah pertama adalah mengklik ikon berbentuk kaca pembesar di sudut kanan atas jendela Arduino IDE. Fungsi utama dari Serial Monitor ini adalah untuk memantau status operasional jaringan dan mengevaluasi komunikasi perangkat dengan *server* Blynk secara *real-time*.



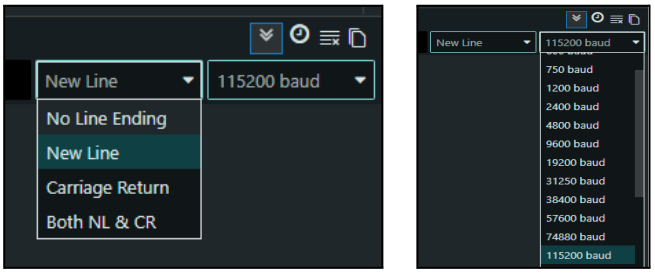
Gambar 6. Ikon Serial Monitor
pada Arduino IDE

Selanjutnya, jendela Serial Monitor akan terbuka pada bagian bawah layar (panel *Output*).



Gambar 7. Tampilan Jendela
Antarmuka Serial Monitor

Sebelum melakukan uji coba sinkronisasi antara perangkat keras dengan aplikasi Blynk, terdapat dua parameter komunikasi yang harus disesuaikan di pojok kanan jendela Serial Monitor. Pastikan pengaturan format baris (*Line Ending*) berada pada opsi New Line dan kecepatan komunikasi (*Baudrate*) berada pada angka 115200 baud, selaras dengan deklarasi Serial.begin(115200) pada fungsi *setup*.



Gambar 25 dan 26. Pengaturan Opsi Line
Ending dan Baudrate

Setelah konfigurasi *baudrate* sesuai, uji coba dapat langsung dilakukan menggunakan *Device* yang telah dikonfigurasi pada aplikasi Blynk. Sebelum memberikan instruksi dari ponsel, disarankan untuk menekan tombol EN (Enable/Reset) pada modul ESP32 secara fisik. Hal ini dilakukan untuk memicu proses *booting* ulang dan memantau log sinkronisasi jaringan pada Serial Monitor guna memastikan ESP32 berhasil terhubung.



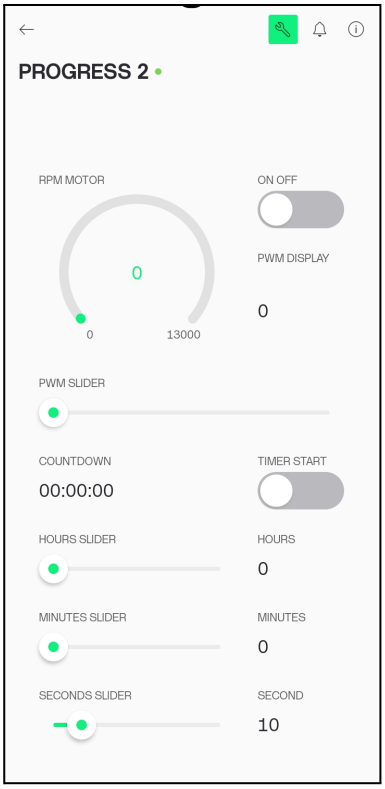
Gambar 8. Log Hasil Pemantauan
Koneksi ke Server Blynk

Buka aplikasi Blynk pada *smartphone*. Indikator visual keberhasilan koneksi *device* ditandai dengan perubahan ikon status perangkat di sudut kanan atas dari warna abu-abu (offline/offline) menjadi warna hijau.



Gambar 9 dan 10. Perubahan Indikator
Status Perangkat Menjadi Online

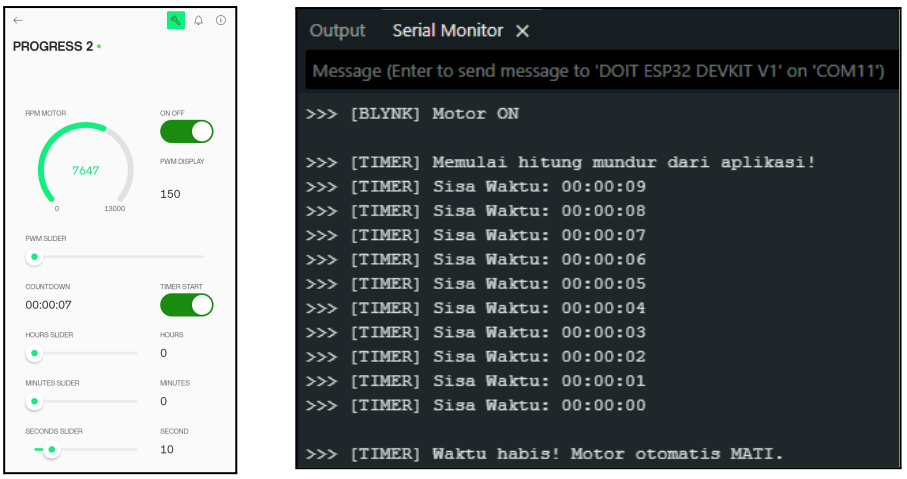
Selanjutnya buka device yang sudah online, pastikan device dalam kondisi offline



Gambar 11. Kondisi Awal Antarmuka
Aplikasi (Motor OFF, Timer 00:00:00)

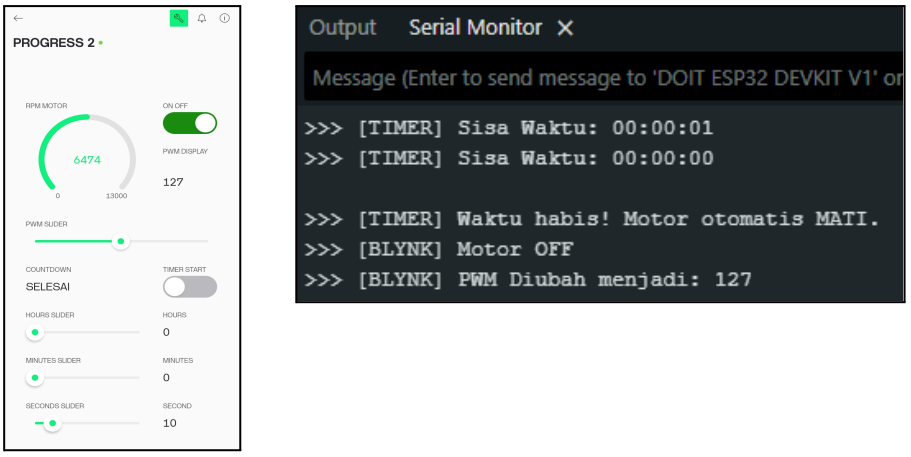
Pengujian fitur utama dilakukan dengan mengatur durasi pada *SECONDS SLIDER* (misalnya: 10 detik), lalu menekan tombol *TIMER START*. Sistem seketika merespons dengan menyalakan motor secara

otomatis (ON). Log Serial Monitor memvalidasi aksi ini dengan mencetak notifikasi penyalaan motor, mengonfirmasi aktivasi hitung mundur dari aplikasi, dan menampilkan laju penyusutan detik secara *real-time*. Pada kondisi ini, jika nilai kecepatan awal berada di angka nol, mikrokontroler secara mandiri menyuntikkan nilai PWM 150 sebagai torsi pemantik.



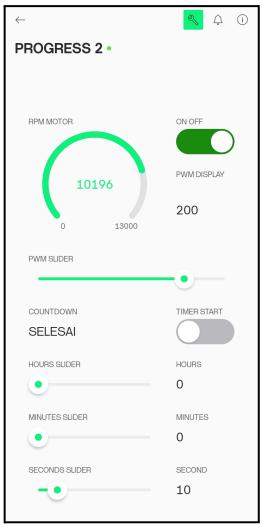
Gambar 12 dan 13. Eksekusi Hitung Mundur dan Pemantauan Log Serial

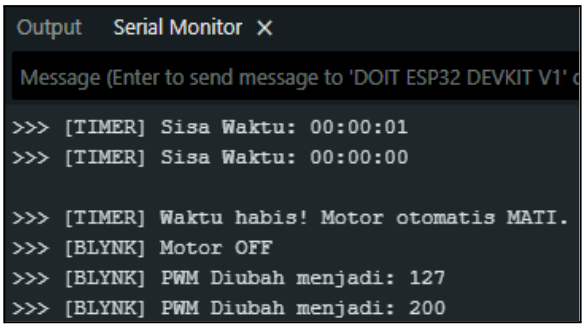
Begitu parameter variabel durasi waktu menyentuh ekuilibrium (habis), arsitektur program secara instan menonaktifkan status variabel putaran mesin. Antarmuka aplikasi menonaktifkan tombol pemicu (*TIMER START*) dan menampilkan konfirmasi "SELESAI", sementara antarmuka Serial Monitor memvalidasi eksekusi akhir dengan log "Waktu habis! Motor otomatis MATI." Meskipun sakelar utama (ON/OFF) tidak dimatikan secara manual oleh pengguna, sistem cerdas ini berhasil memutus siklus PWM secara sepihak (otonom). Pengujian tambahan kemudian dilakukan dengan menurunkan *slider* PWM.



Gambar 14 dan 15. Eksekusi Pemutusan Daya Otomatis dan Modifikasi PWM Pasca-Timer

Uji coba dilanjutkan dengan mendemonstrasikan fleksibilitas kontrol manual di luar pengaruh *timer*. Nilai intensitas PWM kembali dinaikkan secara manual, diikuti dengan pemutusan daya (*motor OFF*) secara manual pula. Parameter konfigurasi waktu (10 detik) yang tertinggal pada *slider* pengatur tidak mengintervensi atau menghalangi eksekusi manual ini, membuktikan bahwa isolasi operasional antara *dynamic timer* dan sakelar mekanis beroperasi dengan sempurna. Setiap modifikasi variabel kecepatan tersebut terekam dengan akurat di log Serial Monitor.





Gambar 16 dan 17. Validasi Operasional Manual Terisolasi Pasca-Eksekusi Timer

c. Analisis Keseluruhan

Berdasarkan serangkaian uji coba yang diinisiasi, skema algoritma hitung mundur berbasis referensi waktu *Non-Blocking Delay* (millis()) terbukti berjalan sangat efisien. Transmisi variabel durasi spesifik dari antarmuka *slider* ponsel ke unit ESP32 dieksekusi secara instan dan dikalkulasi secara presisi di dalam kerangka milidetik. Sistem otomasi ini sukses memitigasi kendala *spam* atau *flooding* lalu lintas data ke jaringan peladen dengan mengunci frekuensi umpan balik (*feedback*) pembaruan tampilan layar hitung mundur (V8) secara eksklusif dalam interval murni satu detik.

Keberhasilan instrumen mikrokontroler dalam mengeksekusi penonaktifan total tegangan PWM secara otonom tepat pada saat variabel durasi waktu habis membuktikan bahwa arsitektur perangkat keras dan lunak beroperasi dalam harmoni mutlak, tanpa indikasi adanya intervensi latensi komunikasi (*network bottleneck*). Pemantauan silang (*cross-monitoring*) ganda yang disediakan melalui

integrasi *Visual Display* di aplikasi *smartphone* dan baris log pada antarmuka Serial Monitor Arduino IDE menjadikan sistem ini sangat transparan dan ideal untuk kebutuhan diagnostik jarak jauh.